ANALISIS SENTIMEN TEKS BERBAHASA INDONESIA MENGGUNAKAN ALGORITMA LONG SHORT-TERM MEMORY (LSTM)



Disusun oleh

Arif Prima Rahadian

(202432107)

Dosen Pengajar

Dr. Dra. Dwina Kuswardani, M.Kom

DEEP LEARNING DALAM SISTEM INFORMASI

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS TELEMATIKA ENERGI

2026/2027

DAFTAR ISI

BAB 1 LATAR BELAKANG MASALAH & EKSPLORASI DATASET 3
1.1 Latar Belakang 3
1.2 Rumusan Masalah 3
1.3 Tujuan 4
1.4 Deskripsi Dataset 4
1.5 Eksplorasi dan Kualitas Data 5
1.6 Bukti Halaman Dataset Kaggle 7
BAB 2 PEMAHAMAN ALGORITMA & PERHITUNGAN MANUAL 9
2.1 Dasar Neural Network 9
2.2 Mekanisme Training 10
2.3 Regularisasi dan Optimasi 11
2.4 Recurrent Neural Network 11
2.5 Long Short-Term Memory 12
2.6 Perhitungan Manual LSTM 13
BAB 3 IMPLEMENTASI PRAKTIKUM & PENJELASAN BARIS KODE 15
3.1 Lingkungan Pengembangan 15
3.2 Import Library 15
3.3 Pemuatan dan Inspeksi Dataset 17
3.4 Preprocessing 20
3.5 Vocabulary, Encoding, dan Tensor 21
3.6 Arsitektur Model 25
3.7 Hyperparameter dan Training 27
3.8 Evaluasi Model 30
3.9 Prediksi pada Data Tanpa Label 32
3.10 Analisis Keterbatasan 33
BAB 4 KESIMPULAN TEORI & PRAKTIKUM 35
DAFTAR PUSTAKA 37

BAB 1
LATAR BELAKANG MASALAH & EKSPLORASI DATASET

1.1 Latar Belakang

Perkembangan layanan digital membuat pengguna semakin sering menyampaikan penilaian melalui komentar, ulasan, dan tanggapan tertulis. Informasi tersebut tidak hanya berisi fakta, tetapi juga opini dan sikap pengguna terhadap produk atau layanan. Analisis sentimen merupakan bagian dari opinion mining yang bertujuan mengidentifikasi orientasi opini di dalam teks, misalnya positif, negatif, atau netral (Pang & Lee, 2008).

Pada data berbahasa Indonesia, klasifikasi sentimen menghadapi variasi ejaan, penggunaan bahasa informal, singkatan, kata tidak baku, serta konteks negasi. Kalimat yang mengandung kata seperti “tidak”, “bukan”, atau “belum” dapat berubah makna apabila kata tersebut dihapus pada tahap preprocessing. Karena itu, pembersihan teks harus menjaga unsur bahasa yang berpengaruh terhadap polaritas sentimen.

Pendekatan pembelajaran mendalam memungkinkan model mempelajari representasi dan pola dari data secara bertahap. Pada klasifikasi teks, urutan kata penting karena makna suatu kata dapat dipengaruhi oleh kata sebelum atau sesudahnya. Long Short-Term Memory (LSTM) merupakan salah satu bentuk recurrent neural network yang dirancang untuk mempertahankan informasi melalui mekanisme gerbang dan cell state, sehingga dapat membantu pemodelan dependensi dalam urutan data (Hochreiter & Schmidhuber, 1997).

Berdasarkan karakteristik tersebut, proyek ini menerapkan LSTM untuk mengklasifikasikan teks berbahasa Indonesia ke dalam tiga kelas sentimen, yaitu negative, neutral, dan positive. Model dibangun menggunakan PyTorch. Eksperimen mencakup pemeriksaan data, preprocessing, pembentukan vocabulary, konversi teks menjadi tensor, pelatihan, serta evaluasi menggunakan metrik klasifikasi.

1.2 Rumusan Masalah

1. Bagaimana mempersiapkan teks berbahasa Indonesia agar dapat diproses sebagai urutan token oleh model LSTM?
2. Bagaimana merancang dan melatih arsitektur LSTM menggunakan PyTorch untuk mengklasifikasikan sentimen negative, neutral, dan positive?
3. Bagaimana performa model berdasarkan accuracy, precision, recall, F1-score, serta confusion matrix pada data berlabel yang digunakan untuk evaluasi?

1.3 Tujuan

1. Mendeskripsikan dataset dan melakukan pemeriksaan kualitas data teks sebelum pelatihan.
2. Menerapkan preprocessing, tokenisasi, vocabulary mapping, padding, dan label encoding untuk membentuk input numerik.
3. Membangun model klasifikasi sentimen berbasis Embedding dan LSTM menggunakan PyTorch.
4. Mengevaluasi performa model dan menginterpretasikan hasil klasifikasi pada tiga kelas sentimen.

1.4 Deskripsi Dataset

Dataset yang digunakan adalah "Dataset for Indonesian Sentiment Analysis" yang dipublikasikan oleh Alvin Kurniawan Hanafie pada Kaggle. Halaman dataset menjelaskan bahwa data ditujukan untuk analisis sentimen dalam bahasa Indonesia, menyediakan pemisahan train, validation, dan test, serta memiliki tiga klasifikasi sentimen: neutral, positive, dan negative. Data Card Kaggle yang terlihat pada dokumentasi juga mencantumkan lisensi Apache 2.0 (Hanafie, n.d.).

Tautan dataset: https://www.kaggle.com/datasets/alvinhanafie/dataset-for-indonesian-sentiment-analysis

Komponen	Informasi
Nama dataset	Dataset for Indonesian Sentiment Analysis
Pemilik Kaggle	Alvin Kurniawan Hanafie (akun: alvinhanafie)
Bahasa	Bahasa Indonesia
Jenis tugas	Klasifikasi sentimen teks
Atribut yang digunakan	text dan sentiment
Kelas target	negative, neutral, positive
Format file	TSV (tab-separated values), berdasarkan nama file dan pemuatan di notebook
Lisensi yang tercantum pada Data Card	Apache 2.0
Tautan	kaggle.com/datasets/alvinhanafie/dataset-for-indonesian-sentiment-analysis

Dalam implementasi, kolom text berfungsi sebagai masukan berupa kalimat atau ulasan, sedangkan kolom sentiment menjadi label target yang hendak diprediksi. Label kemudian diubah menjadi bilangan bulat agar dapat dipakai sebagai target fungsi loss klasifikasi.

Atribut	Tipe konseptual	Keterangan

Atribut	Tipe konseptual	Keterangan
text	Teks/string	Kalimat atau ulasan berbahasa Indonesia yang menjadi masukan model.
sentiment	Kategori/label	Kelas sentimen yang terdiri atas negative, neutral, atau positive.

Struktur split yang tampak pada file dan output notebook adalah 11.000 baris train, 1.260 baris validation, dan 500 baris test masked. Jumlah split 11.000/1.260/500 tersebut sama dengan statistik dataset SmSA yang dicantumkan dalam benchmark IndoNLU (Wilie et al., 2020). Kesesuaian ini mengindikasikan bahwa dataset Kaggle kemungkinan mengemas ulang split SmSA. Akan tetapi, pemeriksaan kesamaan setiap baris atau hash file tidak dilakukan dalam penyusunan dokumen ini, sehingga asal-usul tersebut tidak dinyatakan sebagai hasil verifikasi baris demi baris.

Catatan: Kolom dan jumlah baris di bawah adalah berdasarkan output eksplorasi Colab yang didokumentasikan dalam screenshot. File CSV/TSV asli dan notebook final belum tersedia sebagai lampiran file pada sesi penyusunan ini; cocokkan kembali angka sebelum pengumpulan.

1.5 Eksplorasi dan Kualitas Data

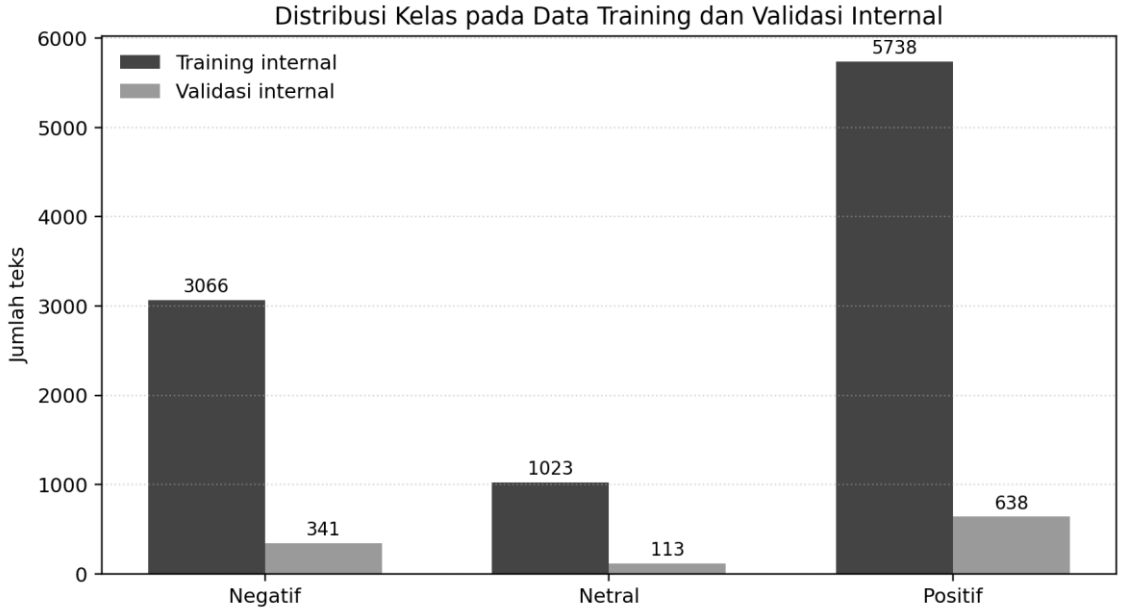
Pemeriksaan awal menunjukkan dua kolom, yaitu text dan sentiment. Output notebook memperlihatkan bahwa file train berisi 11.000 baris dengan distribusi 3.436 negative, 1.148 neutral, dan 6.416 positive. File validation asli berisi 1.260 baris dengan distribusi 394 negative, 131 neutral, dan 735 positive. File test masked berisi 500 baris; label yang terlihat pada kolom sentiment hanya neutral, yang diperlakukan sebagai label placeholder karena file tersebut memang disamarkan.

Split awal	Jumlah baris	Negative	Neutral	Positive	Missing	Teks kosong	Duplikat teks
Train	11.000	3.436	1.148	6.416	0	0	67
Validation	1.260	394	131	735	0	0	0
Test masked	500	0*	500*	0*	0	0	0

**Pada file test masked, nilai neutral sebanyak 500 bukan label sentimen yang diketahui. Nilai tersebut merupakan placeholder dari berkas masked-label dan tidak dipakai sebagai ground truth. Karena itu, distribusi kelas test masked tidak dapat ditafsirkan sebagai distribusi sentimen aktual.*

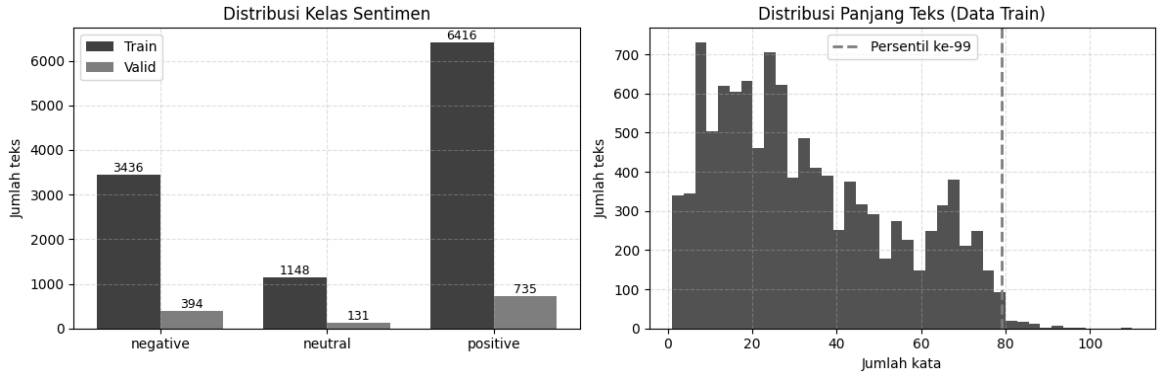
Data training awal memiliki ketidakseimbangan kelas. Kelas positive merupakan kelompok terbesar, sekitar 58,33% dari 11.000 baris, disusul negative sekitar 31,24%,

sedangkan neutral sekitar 10,44%. Ketidakseimbangan ini perlu diperhatikan karena model dapat lebih mudah mempelajari kelas mayoritas. Implementasi mengatasinya dengan class weight pada CrossEntropyLoss.



Gambar 1.1 Distribusi kelas setelah pembersihan dan stratified split pada data training serta validasi internal.

Pada data training, panjang teks memiliki rata-rata 32,9 kata dan median 28 kata. Persentil ke-90 berada pada 67 kata, persentil ke-95 pada 71 kata, persentil ke-99 pada 79 kata, dan maksimum 110 kata. Oleh sebab itu, notebook menetapkan panjang urutan maksimum 80 token. Pemotongan (truncation) hanya terjadi pada sebagian sangat kecil teks training, sekitar 0,03% berdasarkan output yang ditampilkan.



Gambar 1.2 Visualisasi distribusi kelas dan panjang teks yang dihasilkan pada tahap eksplorasi notebook.

Pemeriksaan kualitas data menunjukkan tidak ada missing value dan tidak ada teks kosong pada ketiga file. Pada file training awal ditemukan 67 baris duplikat berdasarkan kolom text. Selain itu, terdapat 14 teks training yang identik dengan teks pada file

validation asli. Notebook menghapus duplikasi dan teks yang beririsan tersebut dari training pool untuk mengurangi risiko kebocoran data.

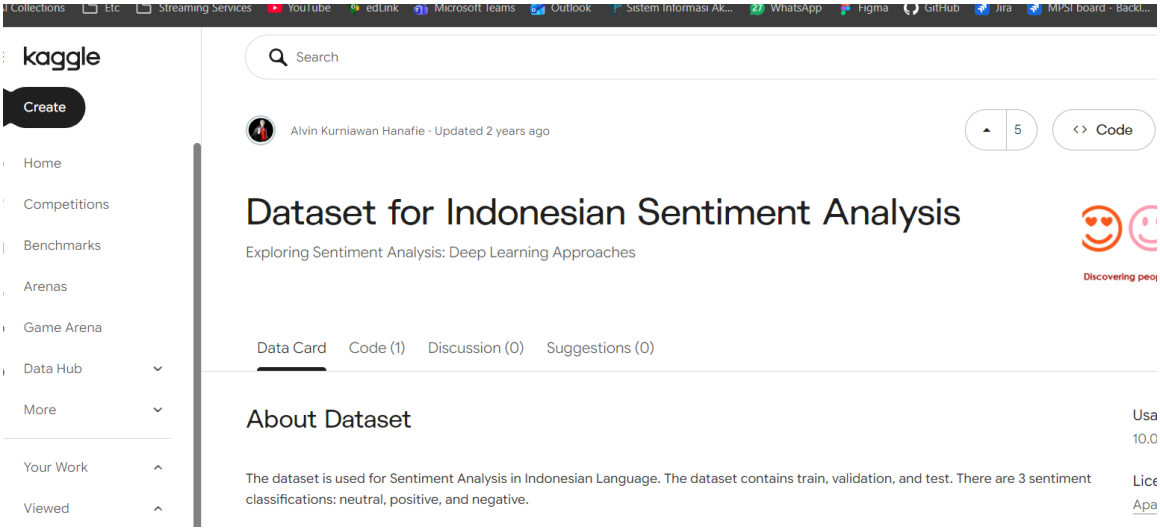
Tahap pembersihan	Jumlah
Data train awal	11.000
Duplikasi teks yang dihapus	67
Teks train yang beririsan dengan validation asli	14
Training pool setelah pembersihan	10.919

Training pool sebanyak 10.919 baris kemudian dibagi secara stratified dengan proporsi 90% untuk training dan 10% untuk validation internal. Hasilnya adalah 9.827 data training dan 1.092 data validation internal. File validation asli yang berjumlah 1.260 baris disimpan sebagai data evaluasi berlabel akhir. File test masked sebanyak 500 baris digunakan untuk inferensi saja.

Subset eksperimen	Jumlah	Peran
Training internal	9.827	Melatih bobot model.
Validation internal	1.092	Memantau loss/accuracy dan early stopping.
Evaluasi berlabel	1.260	Mengukur performa model; berasal dari file validation Kaggle.
Data tanpa label	500	Inferensi; label masked tidak digunakan untuk evaluasi.

1.6 Bukti Halaman Dataset Kaggle

Gambar berikut merupakan screenshot halaman Data Card Kaggle yang menjadi dokumentasi identitas dataset. Pada screenshot terlihat nama dataset, pemilik, deskripsi, informasi bahwa data memiliki split train/validation/test, tiga kelas sentimen, serta lisensi Apache 2.0.



Gambar 1.3 Halaman Data Card Dataset for Indonesian Sentiment Analysis pada Kaggle.

Screenshot ini mendokumentasikan halaman dataset dan informasi yang dicantumkan oleh pengunggah. Bukti tersebut bukan bukti reservasi eksklusif dataset. Status pemilihan dataset untuk kelas tetap perlu disimpan melalui dokumentasi komunikasi atau daftar klaim yang dikelola dosen/asisten.

BAB 2
PEMAHAMAN ALGORITMA & PERHITUNGAN MANUAL

2.1 Dasar Neural Network

Neural network merupakan model komputasi yang tersusun dari unit-unit pemroses yang saling terhubung. Setiap koneksi memiliki bobot, sedangkan setiap neuron dapat memiliki bias. Model memproses masukan melalui operasi perkalian bobot, penjumlahan bias, dan fungsi aktivasi. Dalam pembelajaran terawasi, bobot diperbarui berdasarkan perbedaan antara prediksi dan label sebenarnya (Goodfellow et al., 2016).

2.1.1 Perceptron

Perceptron adalah bentuk sederhana neuron buatan. Untuk masukan x = [x1, x2, ..., xn], perceptron menghitung kombinasi linear $z = w^T x + b$, kemudian menerapkan fungsi aktivasi untuk memperoleh keluaran y = f(z). Bobot w menentukan pengaruh tiap fitur, sementara bias b menggeser nilai aktivasi. Perceptron sederhana dapat digunakan untuk klasifikasi linear, tetapi keterbatasannya mendorong penggunaan jaringan berlapis.

$$z = w^T x + b$$

$$y = f(z)$$

2.1.2 Multi-Layer Perceptron

Multi-Layer Perceptron (MLP) memiliki lapisan masukan, satu atau lebih hidden layer, dan lapisan keluaran. Setiap lapisan melakukan transformasi linear yang diikuti fungsi aktivasi. Hidden layer memungkinkan jaringan mempelajari hubungan non-linear yang tidak dapat direpresentasikan oleh satu perceptron linear. Berbeda dari LSTM, MLP biasa tidak mempertahankan urutan temporal secara eksplisit; masukan biasanya diperlakukan sebagai vektor fitur.

2.1.3 Fungsi Aktivasi

Fungsi	Bentuk	Peran
Sigmoid	$\sigma(z) = 1 / (1 + e^{(-z)})$	Memetakan nilai ke rentang 0 sampai 1; digunakan pada gerbang LSTM.
Tanh	$\tanh(z) = (e^{z} - e^{(-z)}) / (e^{z} + e^{(-z)})$	Memetakan nilai ke rentang −1 sampai 1; digunakan pada kandidat cell state dan transformasi cell state.
ReLU	$\mathrm{ReLU}(z) = \max(0,z)$	Fungsi aktivasi umum pada hidden layer jaringan feed-forward.
Softmax	$\exp(z_i) / \Sigma_j \exp(z_j)$	Mengubah logits menjadi probabilitas kelas; CrossEntropyLoss pada PyTorch menerima logits mentah.

Dalam LSTM, sigmoid digunakan pada forget gate, input gate, dan output gate untuk mengatur seberapa besar informasi dilewatkan. Tanh digunakan untuk membentuk kandidat informasi dan membatasi nilai representasi cell state.

2.2 Mekanisme Training

2.2.1 Forward Propagation

Forward propagation adalah proses meneruskan masukan melalui lapisan model sampai menghasilkan keluaran. Pada proyek ini, token teks terlebih dahulu diubah menjadi indeks, kemudian Embedding Layer mengubah indeks menjadi vektor. Urutan vektor diproses oleh LSTM. Hidden state terakhir diteruskan ke dropout dan fully connected layer untuk menghasilkan tiga logits yang mewakili kelas negative, neutral, dan positive.

2.2.2 Loss Function

Loss function mengukur kesalahan antara logits prediksi dan kelas target. Implementasi menggunakan CrossEntropyLoss untuk klasifikasi tiga kelas. CrossEntropyLoss menerima logits yang belum dinormalisasi dan indeks kelas target. Parameter weight dapat digunakan untuk memberi bobot berbeda pada kelas yang tidak seimbang (PyTorch Contributors, n.d.-a).

$$L = - w_y \log (\exp(z_y) / \Sigma_c \exp(z_c))$$

Karena kelas neutral jumlahnya lebih sedikit, implementasi menggunakan bobot kelas yang dihitung dari frekuensi label training. Berdasarkan ringkasan notebook, bobot untuk negative, neutral, dan positive masing-masing sekitar 1,068; 3,202; dan 0,571. Dengan demikian, kesalahan pada kelas neutral mendapat penalti relatif lebih besar dibanding kelas mayoritas.

2.2.3 Backpropagation dan Optimizer

Backpropagation menghitung gradien loss terhadap parameter jaringan menggunakan aturan rantai. Gradien digunakan optimizer untuk memperbarui bobot agar loss menurun. Optimizer yang dipakai adalah Adam dengan learning rate 0,001. Adam menggabungkan estimasi momen pertama dan kedua dari gradien untuk menghasilkan pembaruan parameter adaptif (Kingma & Ba, 2015).

Dalam training loop, optimizer.zero_grad() menghapus gradien dari iterasi sebelumnya, loss.backward() menghitung gradien, kemudian optimizer.step() memperbarui

parameter. Implementasi juga memakai gradient clipping dengan batas norma 1,0 untuk membatasi besarnya gradien dan membantu kestabilan pelatihan.

2.3 Regularisasi dan Optimasi

2.3.1 Overfitting dan Underfitting

Overfitting terjadi ketika model sangat baik pada data training tetapi performanya lebih rendah pada data yang tidak digunakan untuk memperbarui bobot. Gejalanya dapat terlihat ketika training loss terus turun, sedangkan validation loss mulai meningkat. Underfitting terjadi ketika model belum mampu menangkap pola data dengan baik, sehingga kinerja pada training maupun validation sama-sama rendah.

2.3.2 Dropout

Dropout menonaktifkan sebagian unit secara acak selama training. Tujuannya mengurangi ketergantungan berlebihan antarunit dan membantu generalisasi (Srivastava et al., 2014). Model proyek menggunakan Dropout dengan nilai 0,5 setelah hidden state LSTM dipilih dan sebelum fully connected layer.

2.3.3 Batch Normalization

Batch Normalization menormalisasi aktivasi berdasarkan statistik mini-batch dan memiliki parameter yang dapat dipelajari. Teknik ini dapat membantu stabilitas serta kecepatan pelatihan pada berbagai arsitektur (Ioffe & Szegedy, 2015). Pada implementasi proyek ini, Batch Normalization tidak terlihat digunakan; konsepnya dibahas sebagai bagian teori regularisasi dan optimasi sesuai cakupan materi dosen.

2.3.4 Early Stopping dan Gradient Clipping

Early stopping menghentikan pelatihan ketika metrik validation tidak membaik dalam sejumlah epoch yang ditentukan. Dalam eksperimen, patience ditetapkan 3 dan model terbaik disimpan berdasarkan validation loss. Gradient clipping membatasi norma gradien sebelum optimizer melakukan pembaruan. Keduanya merupakan pengendalian proses training, bukan pengganti evaluasi pada data yang terpisah.

2.4 Recurrent Neural Network

Recurrent Neural Network (RNN) dirancang untuk data sekuensial. Pada setiap time step, RNN menerima input saat ini dan hidden state dari time step sebelumnya. Hidden state membawa ringkasan informasi yang telah diproses. Bentuk dasar pembaruan hidden state dapat ditulis:

$$h_t = tanh(W_x x_t + W_h h_{(t-1)} + b)$$

RNN standar menggunakan satu jalur hidden state untuk meneruskan informasi. Pada urutan yang panjang, gradien yang dihitung melalui banyak time step dapat mengecil atau membesar, sehingga hubungan jangka panjang menjadi lebih sulit dipelajari. LSTM dikembangkan untuk mengatasi keterbatasan tersebut melalui cell state dan mekanisme gerbang.

2.5 Long Short-Term Memory

LSTM diperkenalkan oleh Hochreiter dan Schmidhuber (1997). LSTM mempertahankan cell state yang membawa informasi sepanjang urutan. Gerbang-gerbang yang menggunakan fungsi sigmoid mengatur informasi mana yang dilupakan, ditambahkan, dan dikeluarkan. Mekanisme ini memungkinkan jaringan mengendalikan aliran informasi secara lebih terstruktur daripada RNN sederhana.

2.5.1 Forget Gate

$$f_t = \sigma(W_f \cdot [h_{(t-1)}, x_t] + b_f)$$

Forget gate menentukan bagian cell state sebelumnya yang dipertahankan. Nilai mendekati 0 menunjukkan informasi banyak dilupakan, sedangkan nilai mendekati 1 menunjukkan informasi lebih banyak dipertahankan.

2.5.2 Input Gate dan Candidate Cell State

$$i_t = \sigma(W_i \cdot [h_{(t-1)}, x_t] + b_i)$$

$$\tilde{C}_t = tanh(W_C \cdot [h_{(t-1)}, x_t] + b_C)$$

Input gate mengatur seberapa banyak informasi baru ditambahkan. Candidate cell state menyediakan kandidat nilai informasi baru yang akan dipertimbangkan untuk masuk ke cell state.

2.5.3 Update Cell State

$$C_t = f_t \odot C_{(t-1)} + i_t \odot \tilde{C}_t$$

Cell state baru merupakan gabungan informasi lama yang dipertahankan dan kandidat informasi baru yang lolos melalui input gate. Simbol $\odot$ menyatakan perkalian elemen demi elemen (Hadamard product).

2.5.4 Output Gate dan Hidden State

$$o_t = \sigma(W_o \cdot [h_{(t-1)}, x_t] + b_o)$$

$$h_t = o_t \odot tanh(C_t)$$

Output gate mengatur bagian cell state yang digunakan untuk membentuk hidden state. Hidden state menjadi keluaran pada time step tersebut dan diteruskan ke time step berikutnya atau lapisan klasifikasi. Persamaan ini sejalan dengan formulasi sel LSTM pada dokumentasi PyTorch (PyTorch Contributors, n.d.-b).

2.6 Perhitungan Manual LSTM (Satu Input dan Satu Output)

Perhitungan berikut merupakan ilustrasi scalar untuk memperlihatkan alur komputasi satu time step. Nilai input dan bobot dipilih sederhana agar seluruh operasi dapat diperiksa secara manual. Nilai tersebut bukan bobot hasil training model pada notebook.

Ditetapkan input $x_t = 0,5$, hidden state sebelumnya $h_{(t-1)} = 0,2$, dan cell state sebelumnya $C_{(t-1)} = 0,1$. Vektor gabungan input adalah $[h_{(t-1)}, x_t] = [0,2; 0,5]$. Bobot dan bias ilustratif ditetapkan sebagai berikut:

Komponen	Bobot	Bias
Forget gate	$W_f = [0,4; -0,2]$	$b_f = 0,1$
Input gate	$W_i = [-0,3; 0,5]$	$b_i = 0,2$
Candidate	$W_C = [0,6; 0,1]$	$b_C = -0,1$
Output gate	$W_o = [0,2; 0,3]$	$b_o = 0,0$

Langkah 1: Menghitung Forget Gate

$$z_f = (0,4 \times 0,2) + (-0,2 \times 0,5) + 0,1 = 0,08$$

$$f_t = \sigma(0,08) = 1 / (1 + e^{(-0,08)}) \approx 0,519989$$

Nilai forget gate sekitar 0,520. Artinya, sekitar 52% komponen cell state sebelumnya dipertahankan dalam ilustrasi scalar ini.

Langkah 2: Menghitung Input Gate

$$z_i = (-0,3 \times 0,2) + (0,5 \times 0,5) + 0,2 = 0,39$$

$$i_t = \sigma(0,39) \approx 0,596283$$

Input gate bernilai sekitar 0,596. Nilai ini mengatur proporsi kandidat informasi baru yang akan dimasukkan ke cell state.

Langkah 3: Menghitung Candidate Cell State

$$z_C = (0,6 \times 0,2) + (0,1 \times 0,5) - 0,1 = 0,07$$

$$\tilde{C}_t = \tanh(0,07) \approx 0,069886$$

Kandidat cell state yang dihasilkan bernilai sekitar 0,069886.

Langkah 4: Memperbarui Cell State

$$C_t = (0{,}519989 \times 0{,}1) + (0{,}596283 \times 0{,}069886)$$

$$C_t \approx 0{,}051999 + 0{,}041672 = 0{,}093671$$

Cell state baru bernilai sekitar 0,093671. Nilai ini menggabungkan informasi lama yang dipertahankan dan informasi baru yang ditambahkan.

Langkah 5: Menghitung Output Gate

$$z_o = (0{,}2 \times 0{,}2) + (0{,}3 \times 0{,}5) + 0{,}0 = 0{,}19$$

$$o_t = \sigma(0{,}19) \approx 0{,}547358$$

Langkah 6: Menghitung Hidden State Output

$$\tanh(C_t) = \tanh(0{,}093671) \approx 0{,}093398$$

$$h_t = 0{,}547358 \times 0{,}093398 \approx 0{,}051122$$

Dengan demikian, untuk satu input x_t = 0,5 dan kondisi awal yang ditetapkan, keluaran hidden state pada time step tersebut adalah h_t ≈ 0,051122. Hasil ini menunjukkan bahwa input dan hidden state sebelumnya diolah melalui empat gerbang dan pembaruan cell state sebelum menghasilkan hidden state baru.

Tahap	Hasil
Forget gate f_t	0,519989
Input gate i_t	0,596283
Candidate C̃_t	0,069886
Cell state C_t	0,093671
Output gate o_t	0,547358
Hidden state h_t	0,051122

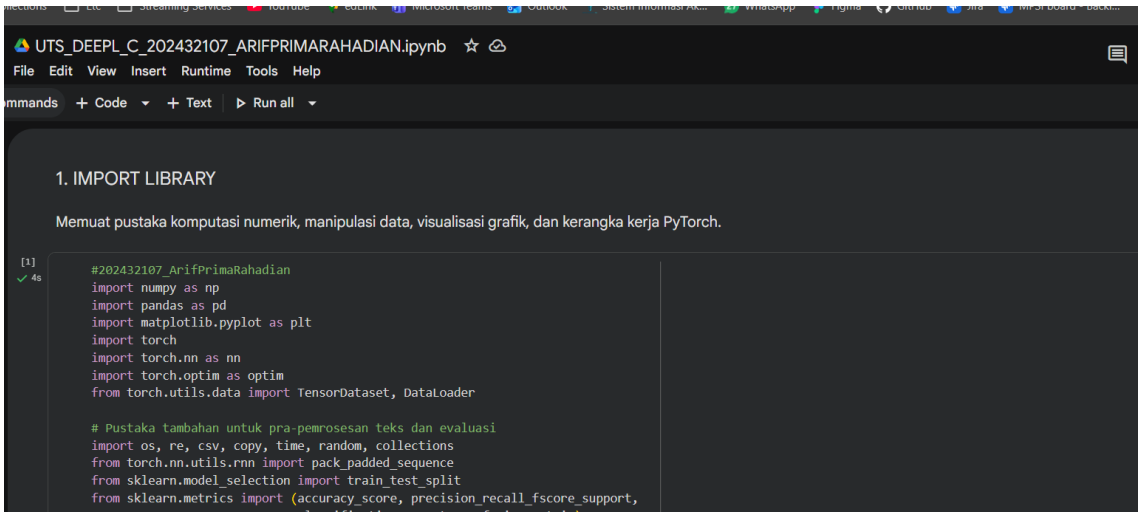
Catatan: Perhitungan scalar ini memenuhi ilustrasi satu input dan satu output pada satu time step. Implementasi PyTorch pada proyek sebenarnya memakai embedding berdimensi 128 dan hidden size 128, sehingga bobot dan state aktual berbentuk vektor/matriks, bukan scalar seperti contoh manual ini.

BAB 3
IMPLEMENTASI PRAKTIKUM & PENJELASAN BARIS KODE

3.1 Lingkungan Pengembangan

Implementasi dilakukan dalam Google Colab menggunakan Python dan PyTorch. Berdasarkan screenshot cell import, output runtime menampilkan PyTorch 2.14.1+cpu dan device CPU. Notebook menetapkan random seed 42 untuk mengurangi variasi hasil dari pengacakan data dan inisialisasi komponen yang menggunakan generator acak. Versi Python tercatat pada metadata notebook menurut ringkasan pemeriksaan sebelumnya, yaitu 3.13.18; nilai tersebut belum terlihat pada output cell yang dilampirkan.

Komponen	Konfigurasi yang terdokumentasi
Lingkungan	Google Colab
Bahasa	Python
Framework	PyTorch 2.14.1+cpu pada screenshot
Device	CPU pada screenshot
Seed	42
Library utama	NumPy, Pandas, Matplotlib, PyTorch, scikit-learn



Gambar 3.1 Cell import library dan konfigurasi seed pada notebook.

3.2 Import Library

Library digunakan untuk manipulasi data, preprocessing teks, pemodelan neural network, pengelolaan batch, evaluasi, dan visualisasi. Potongan kode berikut mengikuti bagian yang terlihat pada screenshot cell import.

3.2.1 Kode import dan konfigurasi awal

```
01  import numpy as np
02  import pandas as pd
03  import matplotlib.pyplot as plt
04  import torch
05  import torch.nn as nn
06  import torch.optim as optim
07  from torch.utils.data import TensorDataset, DataLoader
08  import os, re, csv, copy, time, random, collections
09  from torch.nn.utils.rnn import pack_padded_sequence
10  from sklearn.model_selection import train_test_split
11  from sklearn.metrics import (accuracy_score, precision_recall_fscore_support,
12                               classification_report, confusion_matrix)
13  from matplotlib.colors import LinearSegmentedColormap
14  SEED = 42
15  random.seed(SEED)
16  np.random.seed(SEED)
17  torch.manual_seed(SEED)
18  device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

Penjelasan baris kode:

Baris	Penjelasan
Baris 1	Mengimpor NumPy dengan alias np untuk operasi numerik dan array.
Baris 2	Mengimpor Pandas dengan alias pd untuk membaca serta mengolah tabel dataset.
Baris 3	Mengimpor pyplot untuk membuat grafik eksplorasi dan evaluasi.
Baris 4	Mengimpor pustaka PyTorch sebagai kerangka tensor dan komputasi model.
Baris 5	Mengimpor modul neural network untuk membuat layer dan loss function.
Baris 6	Mengimpor modul optimizer untuk memperbarui parameter model.
Baris 7	Mengimpor TensorDataset dan DataLoader untuk membungkus tensor dan membentuk mini-batch.
Baris 8	Mengimpor fungsi sistem file, regular expression, pembaca CSV/TSV, penyalinan objek, waktu, bilangan acak, dan penghitung frekuensi token.
Baris 9	Mengimpor fungsi untuk mengemas sequence dengan panjang berbeda sebelum masuk ke LSTM.
Baris 10	Mengimpor fungsi pembagian dataset dengan kemampuan stratifikasi.
Baris 11	Mengimpor metrik akurasi serta metrik precision, recall, dan F1 berbasis dukungan kelas.
Baris 12	Mengimpor laporan klasifikasi dan confusion matrix untuk evaluasi.
Baris 13	Mengimpor utilitas colormap untuk visualisasi matriks evaluasi.
Baris 14	Menetapkan seed eksperimen sebesar 42.
Baris 15	Menetapkan seed modul random Python.
Baris 16	Menetapkan seed generator acak NumPy.
Baris 17	Menetapkan seed PyTorch agar eksperimen lebih mudah direproduksi.
Baris 18	Memilih GPU CUDA jika tersedia; jika tidak, memakai CPU.

3.3 Pemuatan dan Inspeksi Dataset

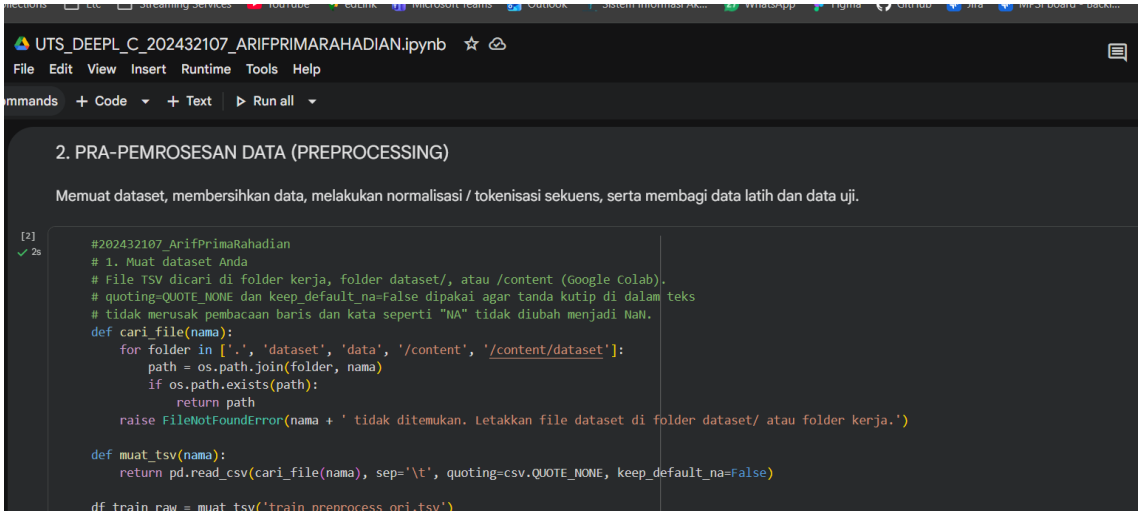
Notebook mencari file TSV pada beberapa kemungkinan folder kerja agar dapat dijalankan dari folder notebook, folder dataset, atau lingkungan Google Colab. Pembacaan menggunakan pemisah tab. Parameter quoting dan keep_default_na dipilih agar teks yang mengandung tanda kutip tidak menimbulkan masalah parsing dan string seperti “NA” tidak otomatis diperlakukan sebagai nilai kosong.

3.3.1 Kode pencarian dan pemuatan file TSV

```
01  def cari_file(nama):
02      for folder in ['', 'dataset', 'data', '/content', '/content/dataset']:
03          path = os.path.join(folder, nama)
04          if os.path.exists(path):
05              return path
06      raise FileNotFoundError(nama + ' tidak ditemukan.')
07
08  def muat_tsv(nama):
09      return pd.read_csv(cari_file(nama), sep='\t', quoting=csv.QUOTE_NONE,
keep_default_na=False)
10  df_train_raw = muat_tsv('train_preprocess_ori.tsv')
11  df_valid_raw = muat_tsv('valid_preprocess.tsv')
12  df_test_raw = muat_tsv('test_preprocess_masked_label.tsv')
```

Penjelasan baris kode:

Baris	Penjelasan
Baris 1	Mendefinisikan fungsi pencarian file berdasarkan nama file.
Baris 2	Memeriksa lima lokasi yang mungkin menjadi folder penyimpanan dataset.
Baris 3	Menggabungkan nama folder dan nama file menjadi path lengkap.
Baris 4	Memeriksa apakah path tersebut benar-benar ada.
Baris 5	Mengembalikan path pertama yang ditemukan.
Baris 6	Menghasilkan error yang jelas jika file tidak ditemukan pada semua lokasi.
Baris 7	Mendefinisikan fungsi untuk membaca file tab-separated values.
Baris 8	Membaca file menggunakan tab sebagai delimiter; QUOTE_NONE menonaktifkan pemrosesan quoting dan keep_default_na=False menjaga string tertentu tidak otomatis menjadi NaN.
Baris 9	Memuat split training awal sebanyak 11.000 baris.
Baris 10	Memuat split validation asli sebanyak 1.260 baris.
Baris 11	Memuat split test masked sebanyak 500 baris.



Gambar 3.2 Screenshot pemuatan dataset TSV pada Google Colab.

3.3.2 Pemeriksaan struktur dan kualitas data

Setelah pemuatan, notebook mencetak nama kolom dan menghitung distribusi label, missing values, teks kosong, serta duplikasi untuk setiap split. Pemeriksaan juga membandingkan teks train dengan teks validation dan test, lalu mengukur panjang kalimat dan frekuensi kata negasi.

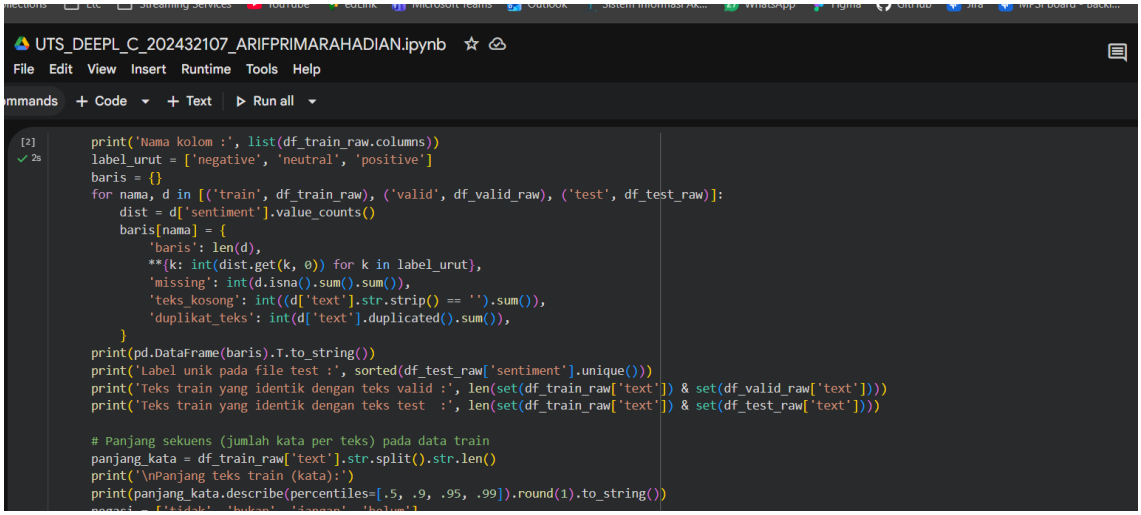
3.3.2 Kode inspeksi dataset

```
01  print('Nama kolom :', list(df_train_raw.columns))
02  label_urut = ['negative', 'neutral', 'positive']
03  baris = {}
04  for nama, d in [('train', df_train_raw), ('valid', df_valid_raw), ('test', df_test_raw)]:
05      dist = d['sentiment'].value_counts()
06      baris[nama] = {
07          'baris': len(d),
08          **{k: int(dist.get(k, 0)) for k in label_urut},
09          'missing': int(d.isna().sum().sum()),
10          'teks_kosong': int((d['text'].str.strip() == '').sum()),
11          'duplikat_teks': int(d['text'].duplicated().sum())
12      }
13  print(pd.DataFrame(baris).T.to_string())
14  print('Label unik pada file test :', sorted(df_test_raw['sentiment'].unique()))
```

Penjelasan baris kode:

Baris	Penjelasan
Baris 1	Menampilkan nama kolom dari data training untuk memastikan skema dataset.
Baris 2	Menetapkan urutan kelas yang digunakan pada tabel dan visualisasi.
Baris 3	Membuat dictionary kosong untuk menyimpan ringkasan setiap split.
Baris 4	Melakukan iterasi terhadap data train, validation, dan test.
Baris 5	Menghitung frekuensi tiap label sentimen.
Baris 6	Membuat record statistik untuk split yang sedang diperiksa.
Baris 7	Mencatat jumlah baris pada split tersebut.

Baris	Penjelasan
Baris 8	Menyimpan jumlah tiap label; kelas yang tidak muncul diberi nilai nol.
Baris 9	Menghitung total nilai kosong pada seluruh kolom.
Baris 10	Menghitung teks kosong setelah spasi di awal/akhir dihapus sementara.
Baris 11	Menghitung duplikasi pada kolom text.
Baris 12	Menutup dictionary statistik.
Baris 13	Mencetak ringkasan statistik dalam bentuk tabel.
Baris 14	Memeriksa nilai label unik di file test untuk mengetahui status label masked.



Gambar 3.3 Screenshot kode inspeksi struktur dan kualitas dataset.

3.4 Preprocessing

Preprocessing dimulai dengan menghapus duplikasi berdasarkan teks dan mengeluarkan teks training yang sama dengan teks pada validation asli. Langkah kedua mengurangi risiko model melihat teks identik pada saat training dan evaluasi. Tokenisasi menggunakan huruf kecil, mempertahankan kata, angka, apostrof atau tanda hubung di dalam token, serta tanda seru dan tanda tanya. Kata negasi dipertahankan karena dapat mengubah orientasi sentimen.

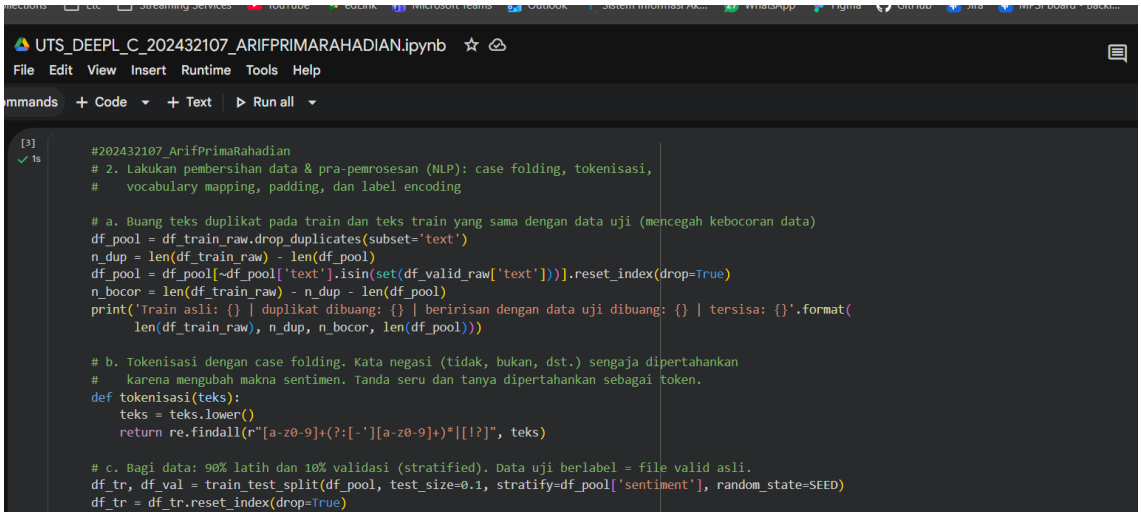
3.4.1 Kode pembersihan, tokenisasi, dan pembagian data

```
01  df_pool = df_train_raw.drop_duplicates(subset=['text'])
02  n_dup = len(df_train_raw) - len(df_pool)
03  df_pool = df_pool[~df_pool['text'].isin(set(df_valid_raw['text']))].reset_index(drop=True)
04  n_bocor = len(df_train_raw) - n_dup - len(df_pool)
05
06  def tokenisasi(teks):
07      teks = teks.lower()
08      return re.findall(r"[a-z0-9]+(?:['-][a-z0-9]+)*|[!?]", teks)
09
10  df_tr, df_val = train_test_split(df_pool, test_size=0.1, stratify=df_pool['sentiment'],
random_state=SEED)
11  df_tr = df_tr.reset_index(drop=True)
```

```
12  df_val = df_val.reset_index(drop=True)
13  df_uji = df_valid_raw.copy()
```

Penjelasan baris kode:

Baris	Penjelasan
Baris 1	Menghapus baris train yang memiliki teks duplikat.
Baris 2	Menghitung jumlah baris duplikat yang dihapus.
Baris 3	Mengeluarkan teks yang muncul di validation asli dan menyusun ulang indeks.
Baris 4	Menghitung jumlah teks train yang beririsan dengan validation setelah duplikasi diperhitungkan.
Baris 5	Baris kosong sebagai pemisah agar struktur kode lebih mudah dibaca.
Baris 6	Mendefinisikan fungsi tokenisasi teks.
Baris 7	Mengubah seluruh huruf menjadi huruf kecil (case folding).
Baris 8	Mengekstrak rangkaian huruf/angka dan tanda ! atau ? sebagai token.
Baris 9	Baris kosong sebagai pemisah.
Baris 10	Membagi training pool menjadi 90% training internal dan 10% validation internal dengan stratifikasi label serta seed yang tetap.
Baris 11	Menyusun ulang indeks dataframe training internal.
Baris 12	Menyusun ulang indeks dataframe validation internal.
Baris 13	Menyalin validation asli sebagai data evaluasi berlabel akhir.



Gambar 3.4 Screenshot preprocessing, penghapusan duplikasi/overlap, tokenisasi, dan stratified split.

Hasil pembersihan menghasilkan 10.919 baris training pool. Dengan stratified split, 9.827 baris digunakan untuk training internal dan 1.092 baris untuk validation internal. File validation asli tetap berjumlah 1.260 dan dipakai untuk evaluasi berlabel. Penamaan variabel X_test pada notebook merujuk pada tensor evaluasi berlabel yang berasal dari file

validation asli; karena itu istilah “data evaluasi berlabel” lebih tepat ketika menjelaskan eksperimen.

3.5 Vocabulary, Encoding, dan Tensor

Vocabulary dibangun dari token pada data training internal saja. Praktik ini mengurangi kebocoran informasi dari validation atau data evaluasi ke proses pembentukan kosakata. Token khusus <pad> diberi indeks 0 dan <unk> indeks 1. Kata yang frekuensinya kurang dari 2 tidak dimasukkan sebagai token tersendiri sehingga saat ditemukan akan dipetakan ke <unk>. Label diubah menjadi integer: negative=0, neutral=1, dan positive=2.

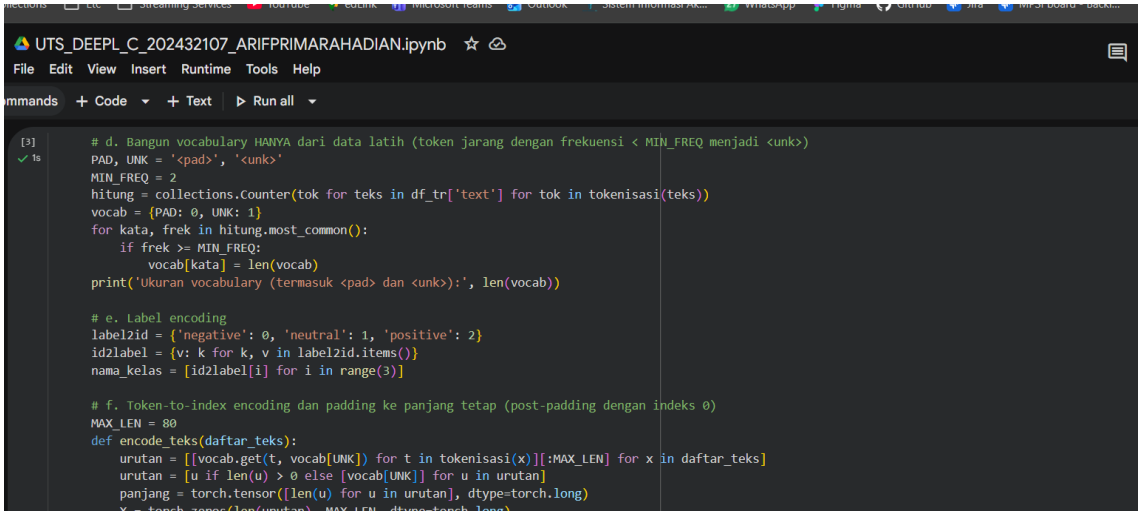
3.5.1 Kode vocabulary, label encoding, dan padding

```
01  PAD, UNK = '<pad>', '<unk>'
02  MIN_FREQ = 2
03  hitung = collections.Counter(tok for teks in df_tr['text'] for tok in tokenisasi(teks))
04  vocab = {PAD: 0, UNK: 1}
05  for kata, frek in hitung.most_common():
06      if frek >= MIN_FREQ:
07          vocab[kata] = len(vocab)
08  label2id = {'negative': 0, 'neutral': 1, 'positive': 2}
09  id2label = {v: k for k, v in label2id.items()}
10  MAX_LEN = 80
11  def encode_teks(daftar_teks):
12      urutan = [[vocab.get(t, vocab[UNK]) for t in tokenisasi(x)[:MAX_LEN]] for x in
daftar_teks]
13      urutan = [u if len(u) > 0 else [vocab[UNK]] for u in urutan]
14      panjang = torch.tensor([len(u) for u in urutan], dtype=torch.long)
15      X = torch.zeros((len(urutan), MAX_LEN), dtype=torch.long)
16      for i, u in enumerate(urutan):
17          X[i, :len(u)] = torch.tensor(u, dtype=torch.long)
18      return X, panjang
```

Penjelasan baris kode:

Baris	Penjelasan
Baris 1	Menetapkan token khusus padding dan unknown.
Baris 2	Menetapkan ambang frekuensi minimum token.
Baris 3	Menghitung frekuensi seluruh token yang berasal dari teks training internal.
Baris 4	Membuat vocabulary awal dengan indeks 0 dan 1 untuk token khusus.
Baris 5	Mengiterasi token berdasarkan frekuensi terbesar.
Baris 6	Memilih token yang memenuhi frekuensi minimum.
Baris 7	Menambahkan token ke vocabulary dengan indeks berikutnya.
Baris 8	Menetapkan pemetaan kelas sentimen ke indeks integer.
Baris 9	Membuat pemetaan balik dari indeks ke nama kelas.
Baris 10	Menetapkan batas panjang sequence sebesar 80 token.
Baris 11	Mendefinisikan fungsi untuk mengubah sekumpulan teks menjadi tensor.

Baris	Penjelasan
Baris 12	Melakukan tokenisasi, pemetaan token ke indeks, penanganan token tak dikenal, dan pemotongan ke panjang maksimum.
Baris 13	Mengganti sequence kosong dengan token unknown agar panjang tidak nol.
Baris 14	Mencatat panjang sequence asli sebelum padding.
Baris 15	Membuat tensor nol untuk menampung sequence dengan panjang tetap 80.
Baris 16	Mengiterasi seluruh sequence hasil encoding.
Baris 17	Mengisi posisi awal tensor dengan indeks token; posisi yang tersisa tetap bernilai nol sebagai padding.
Baris 18	Mengembalikan tensor token dan panjang asli sequence.



Gambar 3.5 Screenshot pembuatan vocabulary, label encoding, dan fungsi padding.

Output notebook menunjukkan vocabulary berukuran 8.492 termasuk token <pad> dan <unk>. Seluruh tensor sequence memiliki panjang tetap 80. Contoh tokenisasi yang ditampilkan antara lain “tempat”, “nya”, “bersih”, “udara”, “segar”, dan “pelayanan”. Kata negasi seperti “tidak”, “bukan”, “jangan”, dan “belum” secara eksplisit dipertahankan.

3.5.2 Konversi ke tensor dan DataLoader

Setelah teks diubah menjadi indeks token, notebook membentuk tensor untuk training, validation, data evaluasi berlabel, dan data tanpa label. Label sentimen diubah menjadi tensor integer. DataLoader digunakan untuk membentuk mini-batch berukuran 64 serta mengacak urutan batch training dengan generator yang seed-nya ditetapkan.

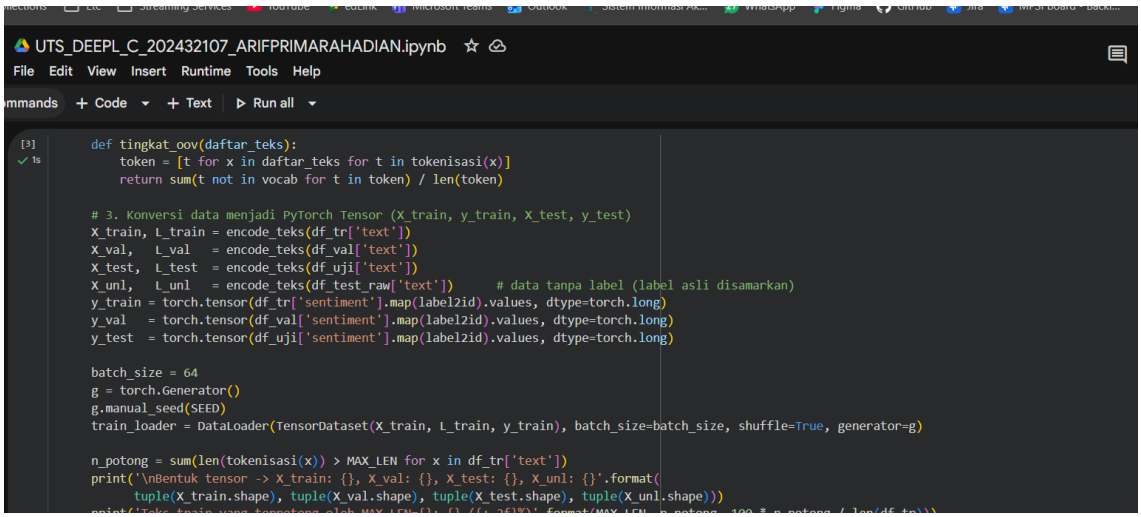
3.5.2 Kode pembentukan tensor dan DataLoader

```
01  X_train, L_train = encode_teks(df_tr['text'])
02  X_val, L_val = encode_teks(df_val['text'])
03  X_test, L_test = encode_teks(df_uji['text'])
04  X_unl, L_unl = encode_teks(df_test_raw['text'])
```

```
05  y_train = torch.tensor(df_tr['sentiment'].map(label2id).values, dtype=torch.long)
06  y_val = torch.tensor(df_val['sentiment'].map(label2id).values, dtype=torch.long)
07  y_test = torch.tensor(df_uji['sentiment'].map(label2id).values, dtype=torch.long)
08  batch_size = 64
09  g = torch.Generator()
10  g.manual_seed(SEED)
11  train_loader = DataLoader(TensorDataset(X_train, L_train, y_train), batch_size=batch_size,
shuffle=True, generator=g)
```

Penjelasan baris kode:

Baris	Penjelasan
Baris 1	Mengubah teks training menjadi tensor indeks dan menyimpan panjang sequence aslinya.
Baris 2	Mengubah teks validation internal menjadi tensor.
Baris 3	Mengubah teks evaluasi berlabel dari file validation asli menjadi tensor.
Baris 4	Mengubah 500 teks masked menjadi tensor untuk inferensi tanpa label.
Baris 5	Mengubah label training menjadi indeks kelas berbentuk tensor integer.
Baris 6	Mengubah label validation internal menjadi tensor integer.
Baris 7	Mengubah label data evaluasi berlabel menjadi tensor integer.
Baris 8	Menetapkan ukuran mini-batch sebesar 64 sampel.
Baris 9	Membuat generator acak PyTorch.
Baris 10	Menetapkan seed generator agar pengacakan batch dapat direproduksi.
Baris 11	Membentuk DataLoader dari tensor training, mengaktifkan pengacakan, dan menggunakan generator yang sudah diberi seed.



Gambar 3.6 Screenshot konversi tensor, distribusi label, dan pembentukan DataLoader.

Variabel tensor	Bentuk	Keterangan
X_train	(9.827, 80)	Token training internal.
X_val	(1.092, 80)	Token validation internal.
X_test	(1.260, 80)	Token data evaluasi berlabel.

Variabel tensor	Bentuk	Keterangan
X_unl	(500, 80)	Token data masked tanpa label.

3.6 Arsitektur Model LSTM

Model disusun dalam urutan Embedding Layer, LSTM, Dropout, dan Fully Connected Layer. Embedding memetakan indeks token ke vektor berdimensi 128. LSTM menerima urutan embedding dengan batch_first=True. Panjang sequence diberikan melalui pack_padded_sequence sehingga padding di bagian akhir urutan tidak diperlakukan sebagai token asli. Hidden state dari lapisan terakhir digunakan sebagai ringkasan sequence dan diteruskan ke Dropout serta lapisan linear yang menghasilkan tiga logits.

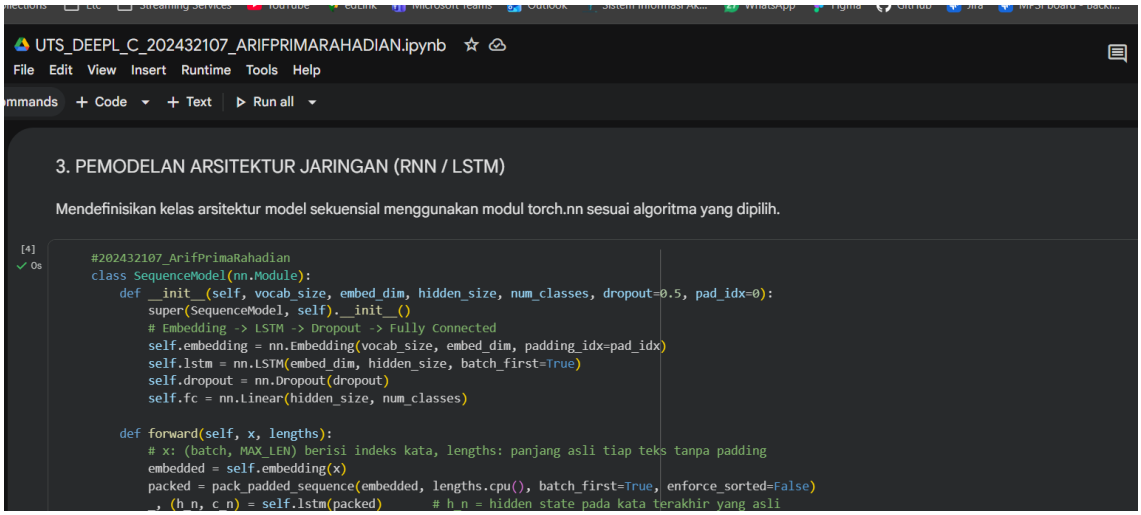
3.6.1 Kode arsitektur SequenceModel

```
01  class SequenceModel(nn.Module):
02      def __init__(self, vocab_size, emb_dim, hidden_size, num_classes, dropout=0.5,
pad_idx=0):
03          super(SequenceModel, self).__init__()
04          self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=pad_idx)
05          self.lstm = nn.LSTM(emb_dim, hidden_size, batch_first=True)
06          self.dropout = nn.Dropout(dropout)
07          self.fc = nn.Linear(hidden_size, num_classes)
08      def forward(self, x, lengths):
09          embedded = self.embedding(x)
10          packed = pack_padded_sequence(embedded, lengths.cpu(), batch_first=True,
enforce_sorted=False)
11          _, (h_n, c_n) = self.lstm(packed)
12          ringkasan = self.dropout(h_n[-1])
13          return self.fc(ringkasan)
```

Penjelasan baris kode:

Baris	Penjelasan
Baris 1	Mendefinisikan class model yang mewarisi nn.Module.
Baris 2	Mendefinisikan konstruktor dengan ukuran vocabulary, embedding, hidden state, jumlah kelas, dropout, dan indeks padding.
Baris 3	Memanggil konstruktor kelas induk PyTorch.
Baris 4	Membuat embedding table dengan indeks padding yang ditentukan.
Baris 5	Membuat satu lapis LSTM dengan dimensi input embedding dan hidden size yang ditentukan; batch_first berarti dimensi batch berada di awal tensor.
Baris 6	Membuat layer dropout dengan probabilitas sesuai parameter.
Baris 7	Membuat fully connected layer yang memetakan hidden size menjadi jumlah kelas.
Baris 8	Mendefinisikan proses forward yang menerima tensor token dan panjang sequence.
Baris 9	Mengubah indeks token menjadi representasi embedding.
Baris 10	Mengemas sequence berdasarkan panjang aslinya agar padding tidak ikut diproses sebagai langkah urutan.

Baris	Penjelasan
Baris 11	Menjalankan LSTM dan mengambil hidden state akhir serta cell state akhir; cell state tidak dipakai langsung pada kepala klasifikasi.
Baris 12	Mengambil hidden state dari layer terakhir dan menerapkan dropout.
Baris 13	Menghasilkan logits tiga kelas melalui fully connected layer.



Gambar 3.7 Screenshot definisi arsitektur SequenceModel berbasis LSTM.

Ringkasan arsitektur yang dicatat pada pemeriksaan notebook adalah Embedding(8.492, 128, padding_idx=0), LSTM(128, 128), Dropout(0,5), dan Linear(128, 3). Model mempunyai satu lapis LSTM satu arah dengan total 1.219.459 parameter. Hidden state akhir yang digunakan adalah h_n[-1].

Komponen	Konfigurasi	Fungsi
Embedding	8.492 × 128	Mengubah indeks token menjadi vektor fitur.
LSTM	input 128; hidden 128; 1 layer; unidirectional	Mempelajari pola dan dependensi urutan kata.
Dropout	0,5	Mengurangi co-adaptation selama training.
Fully Connected	128 ke 3	Mengubah representasi sequence menjadi tiga logits sentimen.

3.7 Hyperparameter dan Proses Training

Hyperparameter dipilih untuk menjaga ukuran model tetap realistis pada runtime CPU. Pelatihan memiliki maksimum 15 epoch, tetapi early stopping menghentikan proses setelah tujuh epoch karena validation loss tidak membaik selama patience yang ditentukan. Model terbaik disimpan berdasarkan nilai validation loss terendah.

Parameter	Nilai	Keterangan

Parameter	Nilai	Keterangan
Epoch maksimum	15	Batas maksimum iterasi penuh dataset.
Learning rate	0,001	Laju pembaruan bobot oleh Adam.
Embedding dimension	128	Dimensi vektor setiap token.
Hidden size	128	Dimensi hidden state LSTM.
Dropout	0,5	Probabilitas unit yang dinonaktifkan saat training.
Batch size	64	Jumlah sampel per batch.
Patience	3	Batas epoch tanpa perbaikan validation loss.
Optimizer	Adam	Optimizer adaptif.
Loss	Weighted CrossEntropyLoss	Cross entropy dengan bobot kelas.
Gradient clipping	1,0	Batas norma gradien.
Seed	42	Seed eksperimen.

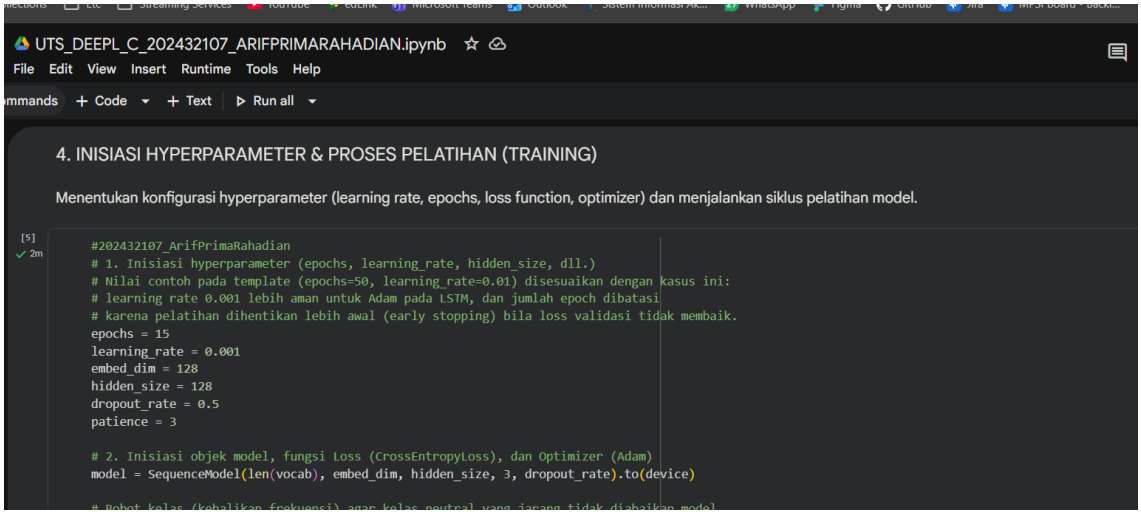
3.7.1 Kode hyperparameter, class weight, dan optimizer

```
01  epochs = 15
02  learning_rate = 0.001
03  embed_dim = 128
04  hidden_size = 128
05  dropout_rate = 0.5
06  patience = 3
07  model = SequenceModel(len(vocab), embed_dim, hidden_size, 3, dropout_rate).to(device)
08  jumlah_kelas = torch.bincount(y_train).float()
09  bobot_kelas = len(y_train) / (3 * jumlah_kelas)
10  criterion = nn.CrossEntropyLoss(weight=bobot_kelas.to(device))
11  optimizer = optim.Adam(model.parameters(), lr=learning_rate)
```

Penjelasan baris kode:

Baris	Penjelasan
Baris 1	Menetapkan maksimum epoch sebesar 15.
Baris 2	Menetapkan learning rate Adam sebesar 0,001.
Baris 3	Menetapkan dimensi embedding sebesar 128.
Baris 4	Menetapkan dimensi hidden state LSTM sebesar 128.
Baris 5	Menetapkan dropout sebesar 0,5.
Baris 6	Menetapkan patience early stopping sebanyak tiga epoch.
Baris 7	Membuat objek model dengan ukuran vocabulary aktual, tiga kelas, dan memindahkan model ke device yang aktif.
Baris 8	Menghitung jumlah contoh untuk setiap kelas pada data training.
Baris 9	Menghitung bobot kelas berdasarkan kebalikan frekuensi kelas, dinormalisasi dengan jumlah kelas.
Baris 10	Membuat CrossEntropyLoss yang menggunakan bobot kelas agar kelas minoritas memiliki kontribusi loss lebih besar.

Baris	Penjelasan
Baris 11	Membuat optimizer Adam untuk memperbarui seluruh parameter model.



Gambar 3.8 Screenshot konfigurasi hyperparameter dan pembobotan kelas.

3.7.2 Alur training loop

Pada tiap epoch, model diaktifkan dalam mode training. Untuk setiap batch, tensor dipindahkan ke device, gradien lama dihapus, logits dihitung melalui forward propagation, loss dihitung menggunakan CrossEntropyLoss berbobot, gradien dihitung dengan backpropagation, gradien dibatasi, lalu optimizer memperbarui bobot. Setelah training satu epoch selesai, model dievaluasi pada validation internal. Validation loss dipakai untuk menentukan apakah state model perlu disimpan dan apakah early stopping harus dijalankan.

3.7.2 Potongan kode training loop

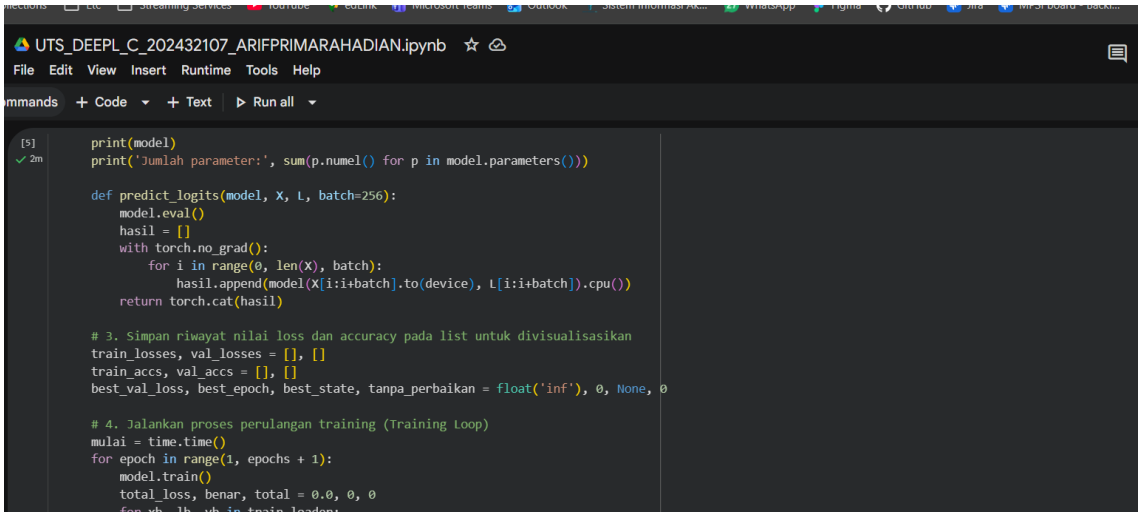
Potongan kode yang terlihat pada dokumentasi screenshot. Langkah lanjutan berupa gradient clipping, optimizer step, validation, dan early stopping dijelaskan berdasarkan ringkasan hasil eksperimen.

```
01  train_losses, val_losses = [], []
02  train_accs, val_accs = [], []
03  best_val_loss, best_epoch, best_state, tanpa_perbaikan = float('inf'), 0, None, 0
04  mulai = time.time()
05  for epoch in range(1, epochs + 1):
06      model.train()
07      total_loss, benar, total = 0.0, 0, 0
08      for xb, lb, yb in train_loader:
09          xb, lb, yb = xb.to(device), lb.to(device), yb.to(device)
10          optimizer.zero_grad()
11          logit = model(xb, lb)
12          loss = criterion(logit, yb)
13          loss.backward()
```

Penjelasan baris kode:

Baris	Penjelasan
Baris 1	Menyiapkan list untuk menyimpan training loss dan validation loss setiap epoch.
Baris 2	Menyiapkan list untuk menyimpan accuracy training dan validation.
Baris 3	Menginisialisasi nilai validation loss terbaik, epoch terbaik, state model terbaik, dan penghitung epoch tanpa perbaikan.
Baris 4	Mencatat waktu awal eksperimen.
Baris 5	Memulai perulangan epoch dari 1 sampai batas epoch maksimum.
Baris 6	Mengaktifkan mode training agar perilaku layer yang bergantung pada mode training berjalan sesuai semestinya.
Baris 7	Menginisialisasi akumulasi loss, jumlah prediksi benar, dan jumlah sampel pada epoch.
Baris 8	Mengiterasi mini-batch yang disediakan DataLoader.
Baris 9	Memindahkan input, panjang sequence, dan label ke CPU atau GPU sesuai device.
Baris 10	Menghapus gradien dari iterasi sebelumnya.
Baris 11	Menghitung logits dengan menjalankan forward propagation model.
Baris 12	Menghitung loss antara logits dan label target.
Baris 13	Menghitung gradien loss terhadap parameter model melalui backpropagation.

Setelah loss.backward(), implementasi menerapkan gradient clipping dengan nilai maksimum 1,0, kemudian memanggil optimizer.step() untuk memperbarui parameter. Sesudah seluruh batch training selesai, notebook melakukan validasi, menghitung loss dan accuracy, menyimpan state model terbaik menggunakan penyalinan state, serta menambah penghitung tanpa perbaikan. Pelatihan berhenti setelah tiga epoch berturut-turut tidak menghasilkan validation loss yang lebih baik.



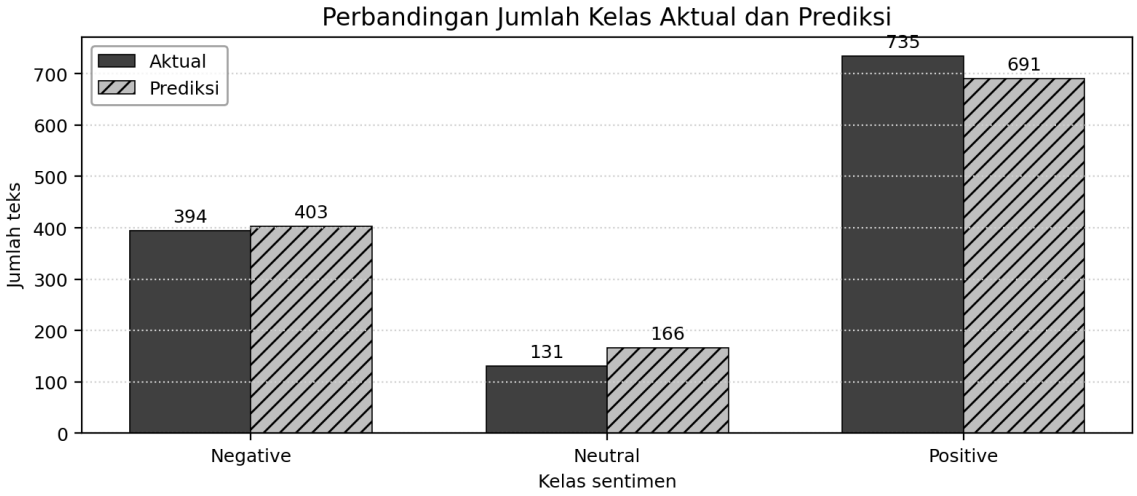
Gambar 3.9 Screenshot bagian training loop dan pencatatan riwayat training.

3.8 Evaluasi Model

Evaluasi menggunakan 1.260 data berlabel dari file valid_preprocess.tsv. Data ini bukan file test masked resmi, sehingga hasil evaluasi harus disebut sebagai evaluasi pada validation asli berlabel yang disimpan terpisah dari training pool. Data test masked sebanyak 500 baris tidak digunakan sebagai ground truth karena labelnya disamarkan.

Output evaluasi pada notebook menunjukkan accuracy sebesar 0,8603, precision macro 0,8043, recall macro 0,8513, dan F1-score macro 0,8240. Nilai weighted F1-score sebesar 0,8629. Accuracy merupakan proporsi prediksi yang tepat dari seluruh data berlabel, sedangkan precision, recall, dan F1-score memberikan gambaran performa per kelas maupun rata-rata. Evaluasi dilakukan pada 1.260 baris dari file valid_preprocess.tsv yang dipisahkan dari proses pembaruan bobot model.

Kelas	Precision	Recall	F1-score	Support
Negative	0,8065	0,8249	0,8156	394
Neutral	0,6687	0,8473	0,7475	131
Positive	0,9378	0,8816	0,9088	735
Macro average	0,8043	0,8513	0,8240	1.260
Weighted average	0,8687	0,8603	0,8629	1.260



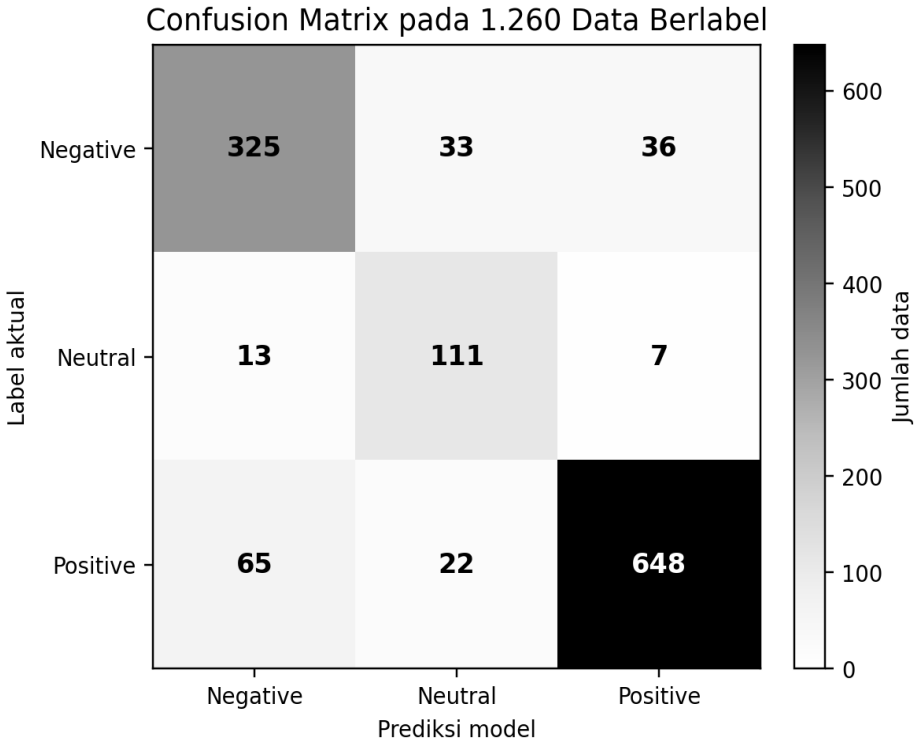
Gambar 3.10 Perbandingan jumlah kelas aktual dan prediksi pada data evaluasi berlabel.

Grafik perbandingan jumlah aktual dan prediksi memperlihatkan jumlah prediksi negative sebanyak 403, neutral sebanyak 166, dan positive sebanyak 691. Dibandingkan jumlah aktual, model menghasilkan prediksi neutral lebih banyak, sedangkan jumlah prediksi positive lebih rendah daripada jumlah aktual. Pola tersebut sejalan dengan precision kelas neutral yang relatif rendah.

Model memperoleh F1-score tertinggi pada kelas positive, yaitu 0,9088. Kelas negative memperoleh F1-score 0,8156. Kelas neutral memiliki F1-score terendah, yaitu 0,7475, dan precision sebesar 0,6687. Nilai precision neutral yang lebih rendah menunjukkan bahwa sebagian prediksi neutral berasal dari teks yang sebenarnya bukan neutral. Hal ini sejalan dengan ketidakseimbangan kelas pada dataset, ketika neutral menjadi kelas dengan jumlah sampel paling sedikit.

Confusion matrix berikut berasal dari output evaluasi notebook pada 1.260 data berlabel. Baris menunjukkan kelas aktual dan kolom menunjukkan kelas prediksi. Nilai diagonal menunjukkan jumlah prediksi yang benar pada masing-masing kelas, sedangkan nilai di luar diagonal menunjukkan kesalahan klasifikasi.

Aktual \ Prediksi	Negative	Neutral	Positive	Jumlah aktual
Negative	325	33	36	394
Neutral	13	111	7	131
Positive	65	22	648	735
Jumlah prediksi	403	166	691	1.260



Gambar 3.11 Confusion matrix hasil evaluasi model pada 1.260 data berlabel.

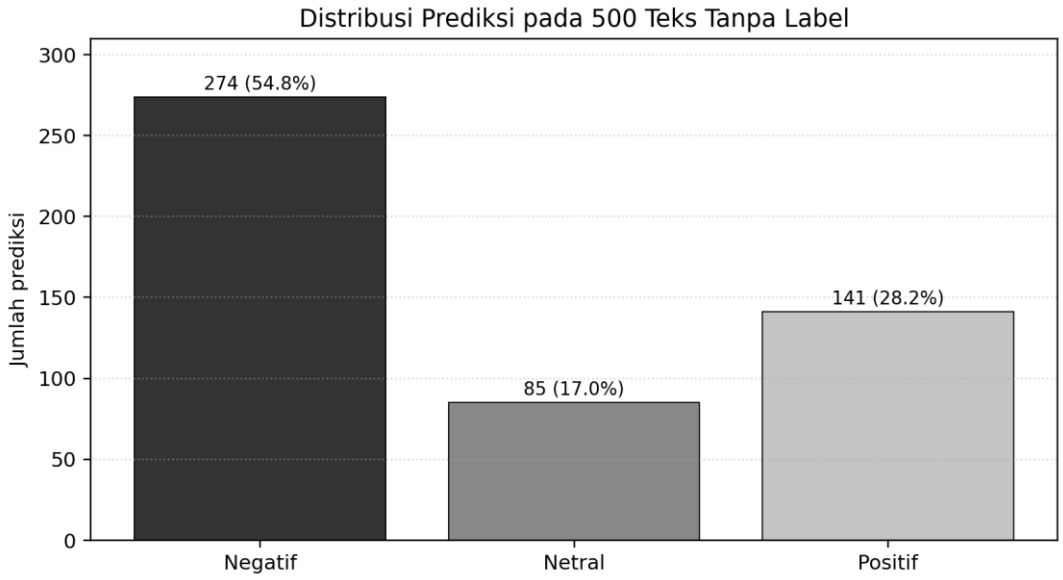
Kesalahan terbesar yang dilaporkan terjadi ketika 65 teks positive diprediksi sebagai negative. Pada kelas neutral, terdapat 111 prediksi benar dari 131 data aktual. Jumlah

prediksi neutral keseluruhan sebanyak 166, sehingga precision neutral adalah 111/166 atau sekitar 0,6687. Temuan ini menunjukkan bahwa model masih memiliki ruang perbaikan untuk membedakan nuansa netral dari opini yang cenderung positif atau negatif.

3.9 Prediksi pada Data Tanpa Label

Selain evaluasi berlabel, model digunakan untuk menghasilkan prediksi pada 500 teks dari file test_preprocess_masked_label.tsv. Karena label asli tidak tersedia, keluaran tahap ini hanya dapat dianalisis sebagai distribusi hasil prediksi dan skor keyakinan model. Tidak tepat menghitung accuracy atau F1 pada data tersebut.

Prediksi sentimen	Jumlah	Persentase
Negative	274	54,8%
Neutral	85	17,0%
Positive	141	28,2%
Total	500	100,0%



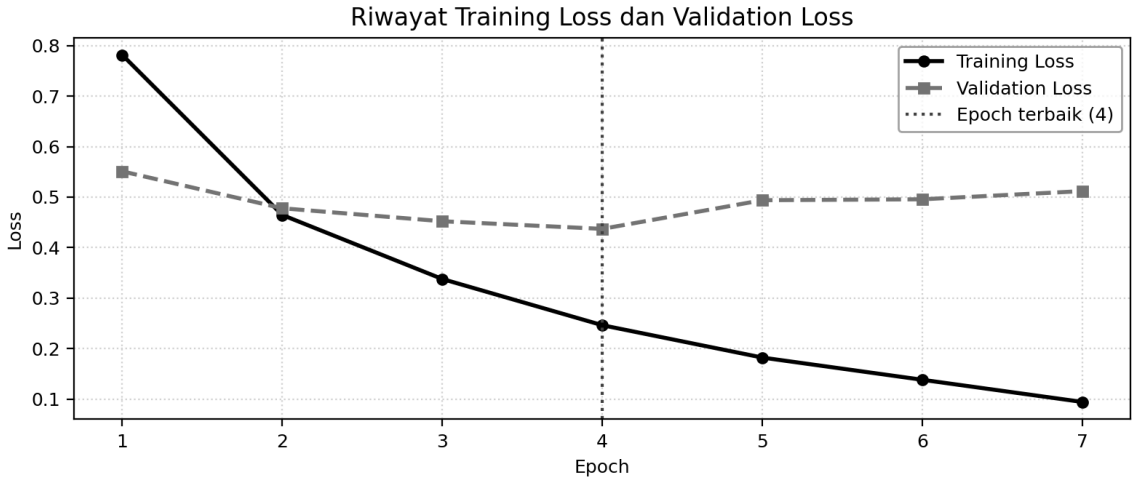
Gambar 3.12 Distribusi hasil prediksi pada 500 teks tanpa label.

Distribusi prediksi memperlihatkan bahwa model paling sering memberikan kelas negative, diikuti positive dan neutral. Distribusi ini bukan distribusi sentimen aktual karena label referensi tidak tersedia. Output evaluasi mencatat rata-rata skor keyakinan 0,9288 pada prediksi yang benar dan 0,7567 pada prediksi yang salah. Pada 500 teks tanpa label, rata-rata skor keyakinan prediksi adalah 0,8640. Skor probabilitas tersebut menggambarkan tingkat keyakinan model, bukan bukti bahwa prediksi pada data tanpa label sudah benar.

3.10 Analisis Training dan Keterbatasan Eksperimen

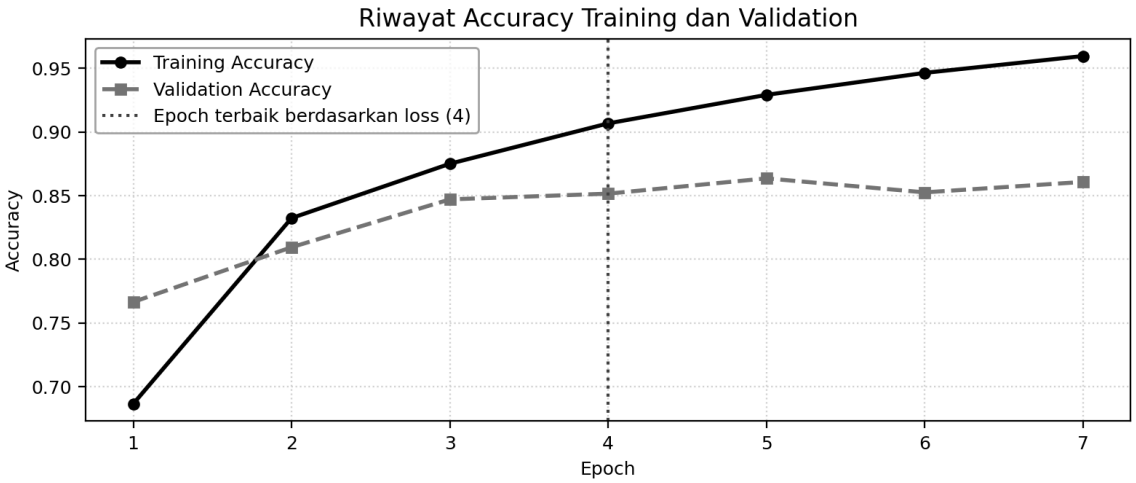
Pelatihan berhenti setelah tujuh epoch karena early stopping. Validation loss terbaik tercapai pada epoch ke-4, yaitu 0,4370, sehingga bobot model dari epoch tersebut dipulihkan untuk evaluasi. Pada epoch ke-7, training loss turun menjadi 0,0942 dan training accuracy mencapai 0,9597, sedangkan validation loss meningkat menjadi 0,5118 dan validation accuracy tercatat 0,8608. Validation accuracy tertinggi selama training adalah 0,8636 pada epoch ke-5. Durasi pelatihan yang dicatat notebook sekitar 108,2 detik pada CPU.

Validation loss meningkat dari 0,4370 pada epoch ke-4 menjadi 0,4938 pada epoch ke-5, kemudian mencapai 0,5118 pada epoch ke-7. Sementara itu, training loss terus menurun. Perbedaan tren tersebut menunjukkan indikasi overfitting. Early stopping dengan patience 3 menghentikan proses pada epoch ke-7 dan menjaga bobot terbaik dari epoch ke-4. Grafik berikut memperlihatkan perkembangan loss dan accuracy selama tujuh epoch.



Gambar 3.13 Kurva training loss dan validation loss selama tujuh epoch.

Keterbatasan	Dampak terhadap interpretasi
Satu seed dan satu konfigurasi eksperimen	Hasil belum menggambarkan variasi performa antar-run atau stabilitas model.
Evaluasi pada file validation asli berlabel	Nilai accuracy/F1 bukan skor pada test resmi benchmark; perbandingan langsung dengan leaderboard tidak tepat.
File test masked tidak memiliki ground truth	Prediksi pada 500 teks hanya dapat dilaporkan sebagai inferensi, bukan evaluasi akurasi.
Ketidakseimbangan kelas	Kelas neutral memiliki support paling rendah dan precision paling rendah.



Gambar 3.14 Riwayat accuracy pada data training dan validation selama tujuh epoch.

BAB 4
KESIMPULAN TEORI & PRAKTIKUM

4.1 Kesimpulan

Proyek ini menerapkan algoritma Long Short-Term Memory untuk klasifikasi sentimen teks berbahasa Indonesia ke dalam tiga kelas, yaitu negative, neutral, dan positive. Dataset Kaggle menyediakan kolom text dan sentiment dengan tiga split. Sebelum pelatihan, data diperiksa untuk mengetahui distribusi kelas, missing values, teks kosong, duplikasi, irisan teks, dan panjang sequence.

Tahap preprocessing meliputi penghapusan data duplikat dan teks yang beririsan dengan validation asli, case folding, tokenisasi, pembentukan vocabulary dari data training, label encoding, serta padding hingga panjang sequence 80 token. Vocabulary yang digunakan berukuran 8.492 token termasuk token khusus. Kata negasi dipertahankan agar informasi sentimen tidak berubah akibat pembersihan yang terlalu agresif.

Arsitektur yang digunakan terdiri atas Embedding Layer berdimensi 128, satu lapis LSTM dengan hidden size 128, Dropout 0,5, dan fully connected layer untuk tiga kelas. Proses pelatihan memakai Adam dengan learning rate 0,001 dan Weighted CrossEntropyLoss. Pembobotan kelas digunakan karena data neutral lebih sedikit daripada negative dan positive. Early stopping dengan patience 3 serta gradient clipping 1,0 digunakan untuk membantu mengendalikan proses training.

Contoh perhitungan manual pada BAB 2 menunjukkan bagaimana forget gate, input gate, candidate cell state, pembaruan cell state, output gate, dan hidden state saling berhubungan dalam satu time step. Pada contoh scalar, input 0,5 menghasilkan hidden state

sekitar 0,051122. Perhitungan tersebut menggambarkan mekanisme internal LSTM secara matematis, sedangkan implementasi sebenarnya memakai vektor embedding dan hidden state berdimensi 128.

Berdasarkan ringkasan evaluasi, model memperoleh accuracy 0,8603 dan macro F1-score 0,8240 pada 1.260 data berlabel yang berasal dari file validation asli. F1-score kelas positive sebesar 0,9088, negative sebesar 0,8156, dan neutral sebesar 0,7475. Kelas neutral masih menjadi kelas yang paling menantang, terutama karena precision-nya lebih rendah. Training accuracy yang lebih tinggi daripada accuracy evaluasi serta kenaikan validation loss setelah epoch tertentu menunjukkan indikasi overfitting.

Sebanyak 500 teks masked juga diproses untuk menghasilkan prediksi. Karena label referensi disamarkan, distribusi 274 negative, 85 neutral, dan 141 positive hanya menggambarkan keluaran model dan tidak dapat digunakan untuk menghitung akurasi. Dengan demikian, proyek ini menunjukkan bahwa LSTM dapat digunakan untuk klasifikasi sentimen tiga kelas pada teks berbahasa Indonesia, tetapi kualitas generalisasi masih perlu ditingkatkan melalui eksperimen lanjutan dan evaluasi yang lebih luas.

4.2 Saran Pengembangan

1. Menjalankan eksperimen dengan beberapa seed untuk memeriksa konsistensi hasil dan melaporkan rata-rata serta simpangan baku metrik.
2. Mengevaluasi beberapa variasi dropout, hidden size, dan panjang sequence dengan hanya menggunakan data training dan validation internal untuk tuning.
3. Menambah atau menyeimbangkan contoh kelas neutral apabila data tambahan berlabel tersedia secara sah.
4. Memeriksa sampel salah klasifikasi secara kualitatif, terutama teks positive yang diprediksi negative dan false positive pada kelas neutral.
5. Menggunakan test set resmi yang memiliki label ground truth apabila tersedia dan diizinkan, sehingga evaluasi akhir dapat dibandingkan secara lebih tepat.
6. Menyimpan notebook, file dataset, versi library, serta instruksi menjalankan ulang eksperimen pada repository GitHub Public sesuai ketentuan dosen.

DAFTAR PUSTAKA

Hanafie, A. K. (n.d.). Dataset for Indonesian Sentiment Analysis [Data set]. Kaggle. https://www.kaggle.com/datasets/alvinhanafie/dataset-for-indonesian-sentiment-analysis

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. Proceedings of the 32nd International Conference on Machine Learning, 37, 448–456. https://proceedings.mlr.press/v37/ioffe15.html

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations. https://doi.org/10.48550/arXiv.1412.6980

Pang, B., & Lee, L. (2008). Opinion mining and sentiment analysis. Foundations and Trends in Information Retrieval, 2(1–2), 1–135. https://doi.org/10.1561/1500000011

PyTorch Contributors. (n.d.-a). CrossEntropyLoss. PyTorch documentation. https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

PyTorch Contributors. (n.d.-b). LSTM. PyTorch documentation. https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html

PyTorch Contributors. (n.d.-c). Adam. PyTorch documentation. https://docs.pytorch.org/docs/main/generated/torch.optim.Adam.html

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15(56), 1929–1958. https://www.jmlr.org/papers/v15/srivastava14a.html

Wilie, B., Vincentio, K., Winata, G. I., Cahyawijaya, S., Li, X., Lim, Z. Y., Soleman, S., Mahendra, R., Fung, P., Bahar, S., & Purwarianti, A. (2020). IndoNLU: Benchmark and resources for evaluating Indonesian natural language understanding. Proceedings of the 1st Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 10th International Joint Conference on Natural Language Processing, 843–857. https://aclanthology.org/2020.aacl-main.85/